Beethoven for VS Code

The Wayne State grid and Claude Code, from one wizard. Manual for version 4.0.0

2026-09-30

- Why
- Getting started
- How it works
- What you can ask, and what you cannot
- Installing and building
- Troubleshooting
- Appendix

> Beethoven for VS Code (called Ashley for VS Code until version 3.0.0) is a VS Code extension for medical students in OpenSpineConsortium projects. It sets up the laptop for the Wayne State HPC grid, keeps a Claude Code worker running there on the student's own subscription, and lets the Claude Code extension on the laptop hand work to it. This manual is the pitch, the first hour, how it works, what you can and cannot ask, installation, and troubleshooting. The design notes and the research behind them are in the repository (`docs/ARCHITECTURE.md`, `docs/RESEARCH.md`).

Why

For Wayne State medical students joining an OpenSpineConsortium project.

You know things no computer scientist on the grid knows. You know why a sacral slope of 9° is implausible, what a transitional vertebra does to a surgeon's count, which patients get the second operation. What you may not have is the year it takes to turn that knowledge into a measurement, a job on 802 CT scans, a figure and a paragraph in a paper. Ashley is built to remove that year.

What Wayne State already gave you

The university runs a grid with more than four hundred compute nodes, NVIDIA H200, L40S and A30 GPUs, hundreds of CPU cores you can hold at once, four terabytes of personal storage and twenty more on the lab's shared volume, and an internet connection from every compute node. Every student with an AccessID may apply for an account. Almost none do, because the entry fee is a week of learning SSH, Slurm, containers and shell quoting, and one misstep runs your job on the login node and gets it killed.

Ashley pays that fee for you. One wizard: your AccessID, one password and one Authenticator code, your GitHub and Hugging Face tokens, and a click in the browser that lets Claude work on the grid as you. From then on the grid never asks you for anything.

What you get

A Claude that lives where the data is. Ashley starts a Slurm job, on a CPU node or one with a GPU, that runs Claude Code inside a container in your grid space, with your own subscription. You talk to it from the Claude Code panel in VS Code, or from a prompt box, or from a phone over ssh if you like. You say *"in my project folder, measure the bi-iliac width on the ten development cases and show me the numbers with units"* and it

reads the manifest, writes the script, runs it, looks at the output, fixes what is wrong and answers. Close the laptop. It is still running.

A conversation that does not forget. Sessions are named. *"In session `thesis`, what did we decide about the exclusion list?"* works next week, from a different node.

Your files, in your editor. One command opens your grid home in VS Code on the compute node where your job runs: Explorer, editor, terminal, and a click on a PNG shows the figure. No X server, no copying files back and forth.

Two Claudes that work together. The Claude Code extension on your laptop gets a handful of tools for the grid. It can plan, and hand the heavy step to the grid worker, and bring the answer back into the panel. You can say *"check the grid, run the pelvic width demo on ten records, and tell me whether the sex difference matches the published value"* in one sentence.

Honesty about limits. When your subscription window is used up, the worker says so, tells you when it resets, and waits. It never uses anyone else's account, never types a token on a command line, and never runs anything heavy on the login node.

Your science stays yours. The lab curates the datasets, the Claude containers and the software in one folder you can read. Everything you build runs in your own grid account, with your own subscription and your own tokens.

Claude in your pocket. Turn on phone access and the Claude app on your phone drives your worker on the grid: check a job from the bus, ask for the next figure between patients.

What becomes possible

- A first-year student with an idea on Monday has ten measured cases by Tuesday and a pre-registered plan by Friday, because the parts that used to take a month (environment, data path, job script, first plot) now take an afternoon.
- A lead runs three projects at once: each has its own worker, its own sessions, its own journal written by the worker at the end of every task.
- A literature question becomes a scoping review pipeline: the worker searches PubMed and OpenAlex with your criteria, logs every inclusion and exclusion, and drafts the PRISMA table, overnight, on a node you are not paying for.
- A model that needs a GPU trains on an H200 while you are in clinic; the loss curve is a PNG in your grid folder when you come back.

What it asks of you

Follow the recipe. Write the question before the number. Freeze the pre-registration. Let the worker run scripts and jobs, and read what it produced yourself: render the

mask, check the spread against the literature, ask what could make a plausible number wrong. Ashley makes the machine easy; the judgement is still the point, and it is yours.

Getting started

Install the extension, run **Ashley: Set up my research environment**, and start a worker. The manual walks through every step with screenshots of what you will see. Your lead can add you to a project card on the board the same week.

Getting started

How the lab works, in one paragraph

The lab curates one folder on the grid's lab volume: the datasets you may use, the Claude containers, and the software. You **read** that folder; you never write to it. All of your science happens in **your own grid account**: your home directory, your jobs, your Claude worker, with your own tokens and your own Claude subscription. Beethoven sets that up and keeps it working; it never asks you to build or install anything under the hood.

Before you open VS Code

1. **Your Wayne State account** (AccessID@wayne.edu). Your lead adds your AccessID to the list of students allowed to use Beethoven.
2. **Accounts, made with your WSU e-mail:** Claude (Pro at the minimum; Max if your lead says so), GitHub, Hugging Face. The lab assumes your Claude account is the one made with AccessID@wayne.edu.
3. **A grid account** (two business days) and **Google Authenticator** for it. The wizard walks you through both.

Install

VS Code, then the extensions **Claude Code**, **Remote - SSH**, and **Beethoven for VS Code** (from the Marketplace, or `beethoven-4.0.0.vsix`: Extensions view, `...`, *Install from VSIX*). Beethoven offers its wizard on first start; or press Ctrl/Cmd+Shift+P and type **Beethoven: Set up my research environment**.

Coming from Ashley for VS Code

Beethoven for VS Code was called Ashley for VS Code until version 3.0.0; it is the same tool under a new name and a new Marketplace listing. What you set up with Ashley carries over:

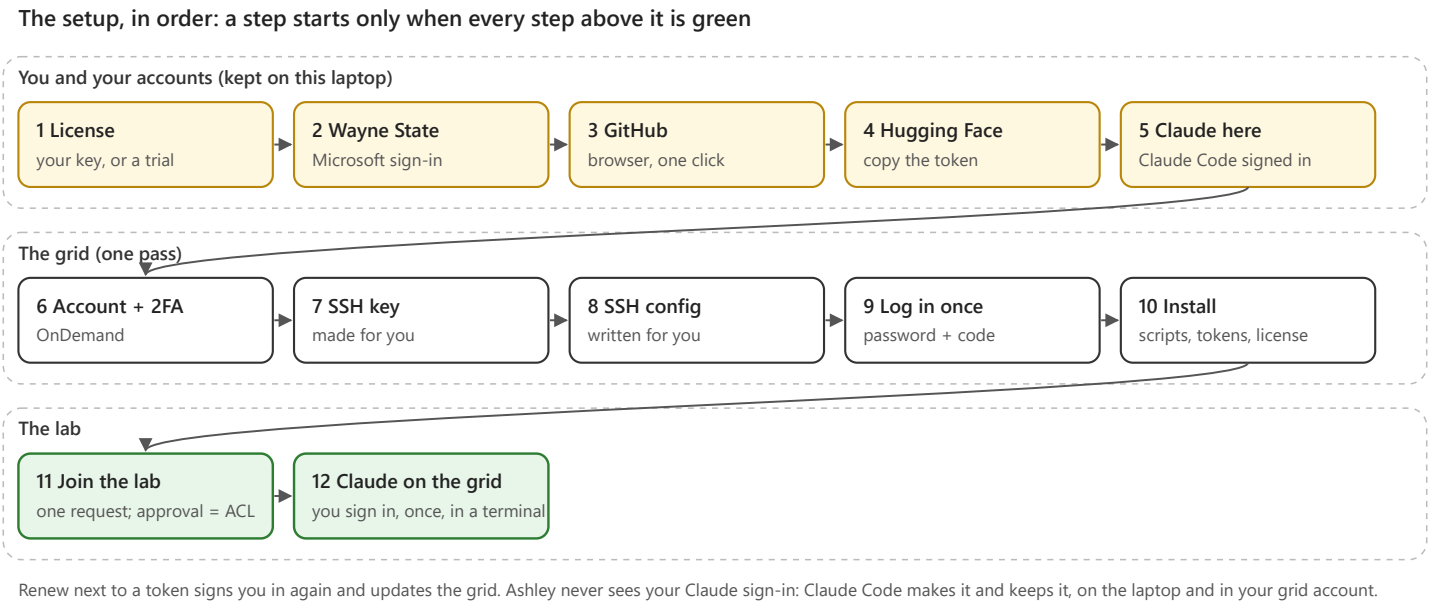
- your settings (Beethoven reads them from their old names until you change them);
- your access key, your Wayne State sign-in record and your SSH setup on this laptop;
- everything in your grid account. The first grid install of this version renames your grid folder from `~/.ashley` to `~/.beethoven`, in one step, while no worker runs, and leaves the old name pointing at the new folder. Your Claude sign-in on the grid, your license and your workers' records move with it, so you do not sign in again there.

Two things are asked again, once: the agreement (a new version, shown in the signing dialog), and your GitHub and Hugging Face tokens, because VS Code keeps each extension's stored tokens apart. Run the wizard: its GitHub step is one click in the browser and its Hugging Face step one copy.

Then uninstall Ashley for VS Code (Beethoven offers to do it). While both are installed, each puts its own version of the grid scripts back into your grid account.

The setup panel

One panel: the steps on the left with their state, the current question on the right, a progress bar on top, the log one click away. The order is strict: a step does not start until every step above it is green. Your license and your accounts come first and their tokens are kept on this laptop; the grid comes next and receives everything in one pass; joining the lab comes after that, and your Claude sign-in on the grid is last, because it happens inside the lab's container.



Renew next to a token signs you in again and updates the grid. Ashley never sees your Claude sign-in: Claude Code makes it and keeps it, on the laptop and in your grid account.

The wizard's steps

Step	What you see	What you do
1 Sign in with Wayne State	your browser opens the WSU sign-in	sign in with AccessID@wayne.edu. Only AccessIDs the lab has approved continue; if yours is not, the next step sends you to the request page
2 Access	a spinner, then your key's end date; or a button *Ask for access*	nothing, normally: Beethoven asks the lab's access service for a key in your name and keeps it. Not approved yet? *Ask for access* opens the lab's page; when your lead approves you, run this step again. A key your lead sent can be pasted here too (a Full Edition key opens its own agreement)
3 GitHub	your browser opens GitHub	sign in and approve; the token arrives by itself
4 Hugging Face	a button opens the token page, pre-filled	press *Create token*, then *Copy*; the panel takes it
5 Claude on this laptop	a check of Claude Code's own sign-in	sign in to Claude Code with the account you made with your WSU e-mail (Claude Pro at the minimum). Beethoven only looks at whether you are signed in
6 Grid account and Authenticator	a menu (the panel says whether you are on the campus Wi-Fi or the VPN)	if you have no grid account, apply; on OnDemand open 2FA Setup and scan the QR code; then *continue*
7 SSH key	a spinner	nothing
8 SSH configuration	a spinner	nothing
9 Log in to the grid once	a password field, then a code field	your grid password, then the six-digit Authenticator code. Once; the grid never asks again
10 Install on the grid	a progress line	nothing: scripts, your GitHub and Hugging Face tokens and your license go to the grid in one pass
11 Join the lab	a note field, only when everything above is green	a sentence for your lead (project, lead). One request; your lead approves it in Beethoven, and from then on you read the lab folder and

Step	What you see	What you do
		your workers use the lab's Claude container
12 Sign in to Claude on the grid	a terminal on the grid	Claude Code signs you in inside your own grid account: click the link it prints, sign in with the same Claude account, press Authorize, paste the code back. Once a month or so; Beethoven says when. Beethoven never sees this sign-in

The first time Beethoven runs it shows the Trial Edition license agreement and the privacy statement in a signing dialog. You read the full text (scroll to its end), tick four statements that must be true, choose whether Beethoven may send its maintainer error reports, type your name, and choose **Accept and Continue**. That click forms the agreement; nothing happens before it. Beethoven keeps a record of what you accepted (the version, the text, what you ticked, your name, the time) in `~/.beethoven-local` (on a laptop that ran Ashley, in `~/.ashley-local`), where you can read, copy and print it, and **Beethoven: License** opens the text again at any time. The Trial Edition is free: nothing is paid, nothing converts to a paid license, and it can be withdrawn or ended with notice. The agreement sends disputes to Michigan courts after 60 days of written notice, with a class-action waiver and a jury waiver. If a paid Full Edition is offered later, registering its key shows its own agreement in the same dialog; nothing obliges you to buy it. An error report says which check failed and why, with names, paths and anything secret replaced; never your prompts, files, tokens or AccessID. Say no in the dialog, or later in Settings, and nothing is ever sent. **Beethoven: Error reports** shows exactly what a report contains.

Every step has a button next to it. **Renew** next to a token signs you in again and updates the grid copy in the same click; Beethoven checks your tokens once a day and offers Renew when one stops working. **Run again** reruns any other step.

The Beethoven view (the Beethoven icon in the activity bar) shows the same things as rows: your tokens under *Tokens on this laptop*, *Claude on this laptop*, *Claude on the grid*, and *Wayne State sign-in* when that is what is missing. When a row says **missing**, **expired** or **not signed in**, click it: the same sign-in the wizard runs for that step opens, and nothing else changes. The troubleshooter offers the same button next to a failing check.

Your first worker and your first question

1. **Beethoven: Start a Claude worker on the grid** (after your lead approved step 10): name `default`, CPU node, 24 hours.

2. **Beethoven: Ask the grid Claude**: *"Create a folder hello in my home with a README that says which node you are on and what Python version you have."* The answer streams in; tool calls appear as italics.
3. **Beethoven: Open my grid files on a compute node**: a new VS Code window opens on the node. It shows a banner that the OS is unsupported: expected on the grid's CentOS 7 (Wayne State supplies the compatibility libraries); dismiss it.

From your phone

Beethoven: Phone access turns on Claude Code's Remote Control on your worker. The first time, your browser opens claude.ai: sign in with the same account, press *Authorize*, press *Copy* next to the code the page shows; Beethoven hands it to the worker. The session then appears in the Claude app on your phone (and at claude.ai/code) whenever the worker runs, exactly like `/remote-control` in a Claude Code session on your laptop. The sign-in lasts about 30 days; the same command renews it.

Talking to it from Claude Code

Beethoven: Connect the Claude Code extension to the grid registers the bridge (the wizard does it for you). Then, in the Claude Code panel: *"Check the grid status and tell me whether my worker is idle,"* or *"Ask the grid Claude, in session `demo`, to run the pelvic width demo on ten records and report the mean."*

Starting over, or showing a friend

Beethoven: Reset this laptop (emulate a fresh install) forgets, as you choose, the tokens, the key and config block, the settings, the Claude Code registration, and Beethoven on the grid. After a reload, VS Code looks the way it does to a brand-new student. For a brand-new VS Code as well: `code --user-data-dir %TEMP%\fresh-vscode --extensions-dir %TEMP%\fresh-ext`.

Every day after that

- Start a worker when you sit down, or keep one running for a day.
- Name your sessions after the project; a session remembers its folder.
- Let the worker write scripts and Slurm job files; you read them; you say "submit".
- If something is off: the Beethoven item in the status bar turns red; **Troubleshoot** finds the broken piece and repairs what it can; **Show log** has the details.

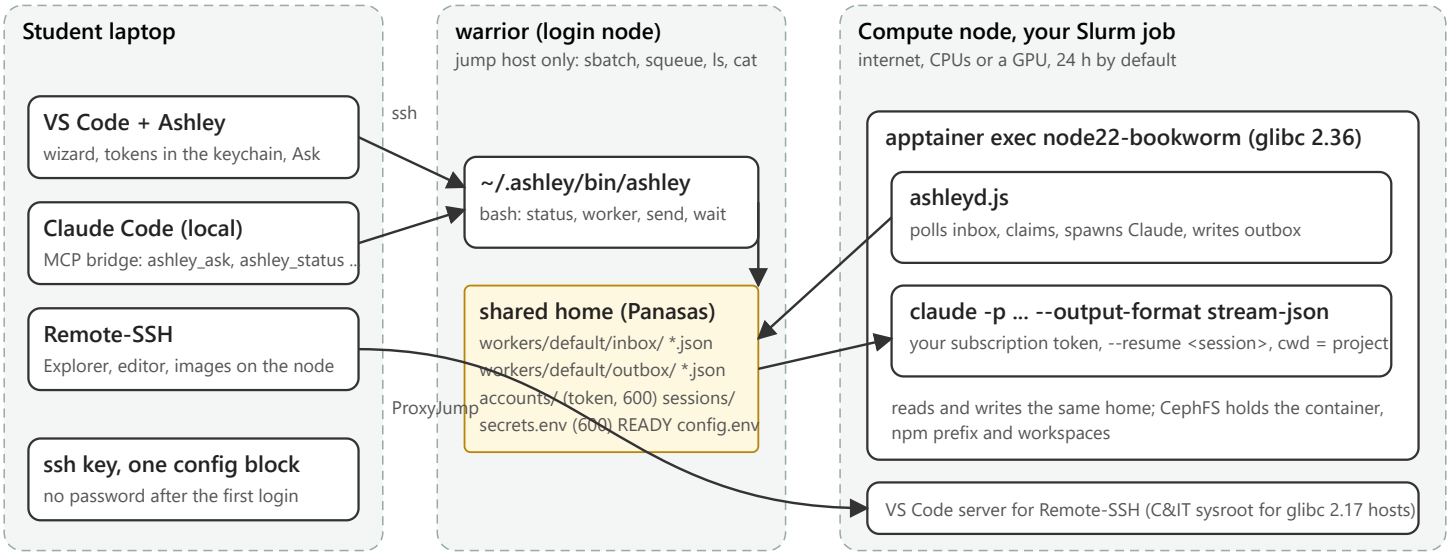
How it works

Ashley is three small things that agree on one file format.

Piece	Runs on	Language	Job
VS Code extension (`extension/`)	the student's laptop, inside VS Code	TypeScript	wizard, secrets, Remote-SSH, the Ashley view, "Ask", MCP registration
MCP server (`extension/dist/mcp-server.js`)	the laptop, spawned by Claude Code	TypeScript (bundled)	gives the local Claude the `ashley_*` tools
Grid CLI + daemon (`grid/`)	the grid: `ashley` on the login node, `ashleyd.js` inside a container on a compute node	bash, Node	install, workers, the request queue, Claude Code itself

Everything the laptop does on the grid is `ssh wsu-grid ~/.ashley/bin/ashley <verb>`. There is no port, no tunnel and no server on the laptop; nothing on the login node beyond `sbatch`, `squeue`, `cat`, `ls` and a half-second sleep loop.

Topology



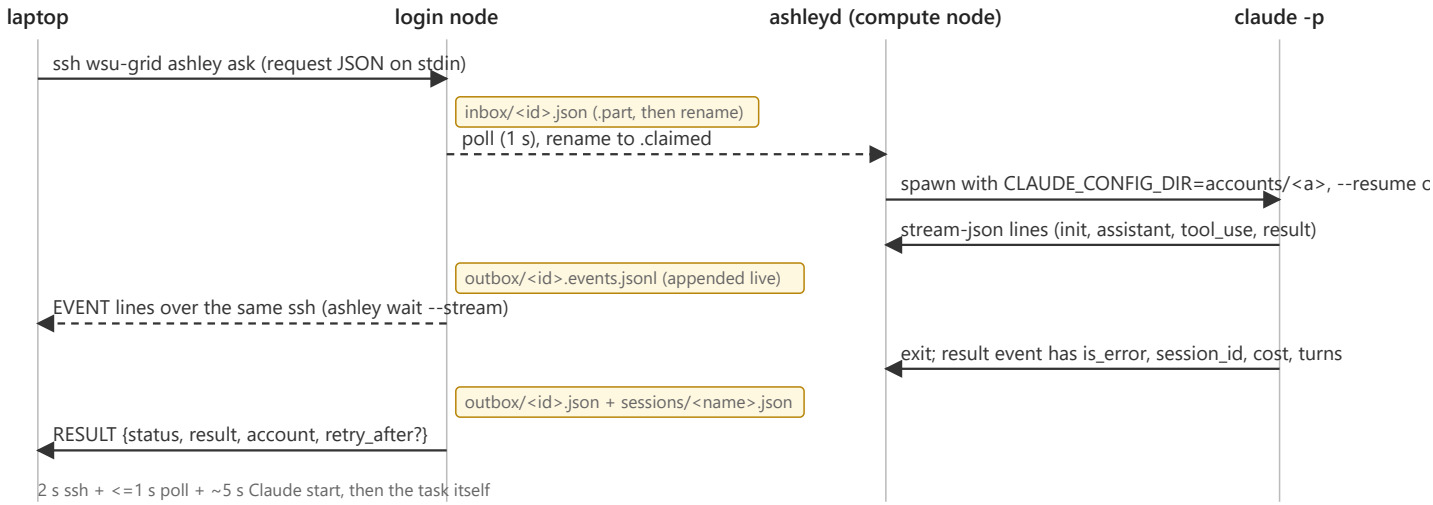
The three places and what runs in each; every laptop action is one ssh to the login node.

```
laptop                          warrior (login node)              compute node (Slurm job)
──────                          ────────────────────              ────────────────────────

VS Code — Ashley ext ──ssh──▶ ~/.ashley/bin/ashley ──files──▶  apptainer exec node22-bookworm
Claude Code — MCP ─┘           (bash: install, status,          └─ node ashleyd.js
Remote-SSH ──────────ProxyJump──────────────────────────────▶     └─ claude -p ... (2.1.283)
                               sbatch / squeue                     watches  workers/<w>/inbox
                                                                   writes   workers/<w>/outbox
                               shared home: ~/.ashley/{accounts,sessions,workers}
                               CephFS ROOT: containers/, npm/, workspaces/
```

The login node is a jump host. VS Code Remote-SSH connects to `voh3.nas.grid.wayne.edu` (any node where the student holds a job) through `ProxyJump wsu-grid`; C&IT forbids the VS Code server on `warrior` and kills it there. Compute nodes have internet, so Claude Code, `npm`, Hugging Face and Zenodo all work from inside the job.

The request queue



One request, from the laptop's ssh to the RESULT line.

A request is a JSON file. Its life:

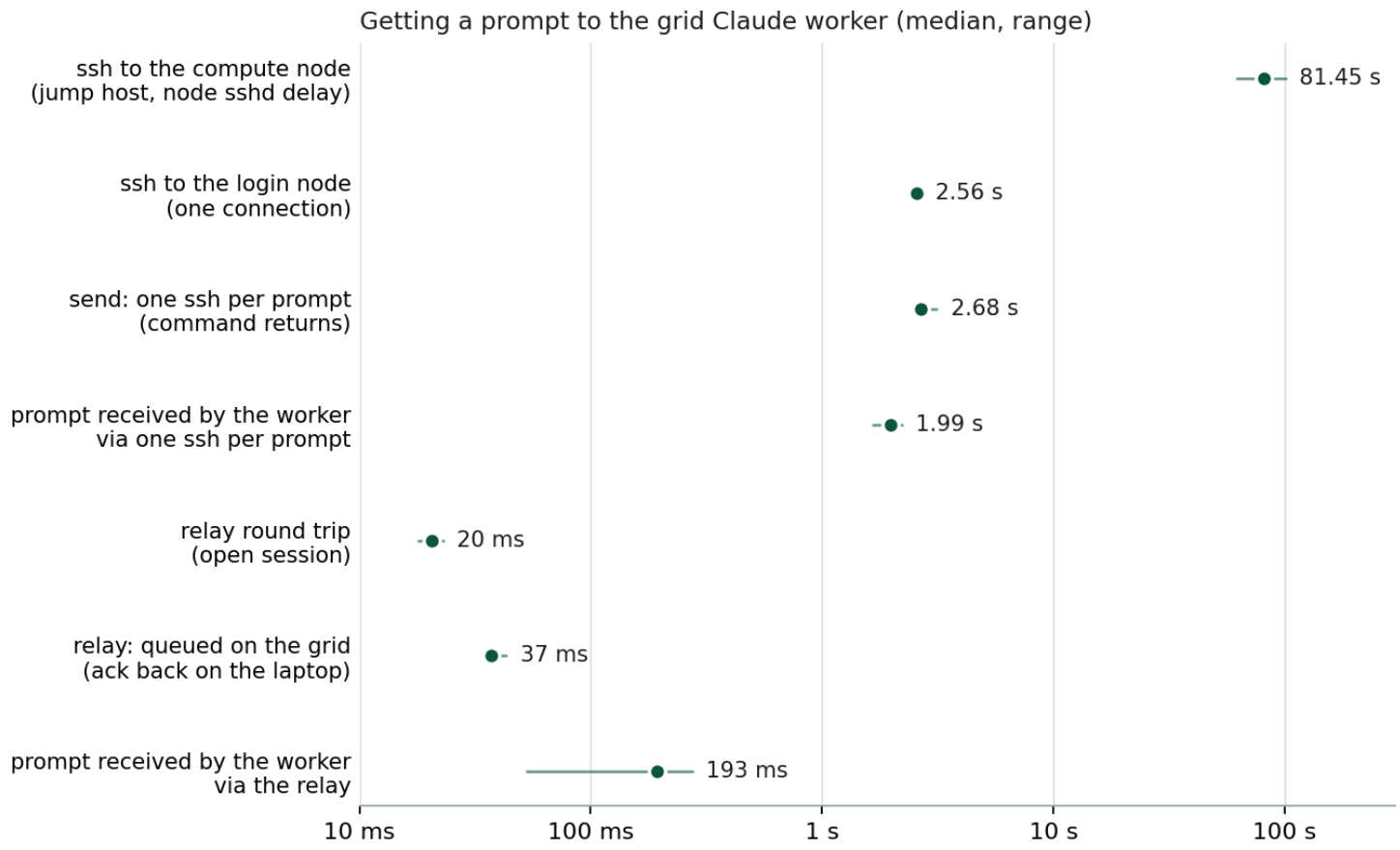
1. The laptop writes `~/.ashley/workers/<w>/inbox/<id>.json` through `ashley send` (stdin, `.part` then rename, so the daemon never reads a half-written file).
2. `ashleyd.js` polls the inbox every second, claims the file by renaming it to `<id>.json.claimed` (an atomic operation on the shared filesystem; two daemons cannot both win).
3. It picks a credential (below), resolves the session, and spawns `claude -p <prompt> --output-format stream-json --verbose --permission-mode <mode> --max-turns N [--resume <sid> | --session-id <uuid>]` in the session's working directory, with `CLAUDE_CONFIG_DIR=~/.ashley/accounts/<account>`.

4. Every stream-json line is appended to `outbox/<id>.events.jsonl` as it arrives. `ashley wait --stream` tails that file to the laptop as `EVENT {…}` lines, so VS Code shows tool calls live.
5. On exit, the daemon writes `outbox/<id>.json` (`.part` then rename): status, result text, session id, account, cost, turns, duration, node, job. `ashley wait` prints it as `RESULT {…}`.

How a prompt travels, and how fast

The laptop keeps **one ssh session** to the login node open, running `ashley relay`. A prompt is one line on that session's stdin; the relay writes the inbox file and answers `QUEUED`; the daemon polls the inbox every 250 ms; results and events come back on the same session as `RESULT` / `EVENT` lines. Measured on 2026-09-28 (Windows laptop, skew-corrected; `docs/research/`):

Path	Median	Range
one ssh connection to the login node	2.6 s	2.4 to 2.7 s
prompt received by the worker, one ssh per prompt (the first design)	2.0 s	1.6 to 2.2 s
relay round trip	20 ms	18 to 23 ms
relay: queued on the grid, ack on the laptop	37 ms	35 to 43 ms
prompt received by the worker via the relay	**0.19 s**	0.05 to 0.28 s
ssh straight to the compute node (for comparison)	about 25 s	the node's sshd delay



Windows 11 laptop, Wayne State grid, 2026-09-28. Receipt = worker's claim time minus the laptop's send time.

Latency of the prompt paths

So the relay is roughly ten times faster to receipt and the remaining 0.1 to 0.3 s is the daemon's poll interval. It is "UDP-like" in the sense that matters (no per-message handshake, fire-and-forget `send` with an ack in 40 ms) while staying reliable: the file is the message, and a dropped session loses nothing that was already queued. If the session drops, the next call reconnects; every call also falls back to a fresh ssh. True UDP to the node is impossible (the laptop cannot reach compute nodes) and unnecessary. Claude Code's own start-up (about 5 s cold) now dominates the time to a first token; a warm `--input-format stream-json` process per session is the next lever if that ever matters.

A queued request survives a daemon restart, a requeued job or a closed laptop: the daemon moves any `.claimed` file back to the inbox when it starts. An interactive ssh to the login node or the node (Remote-SSH, `Ashley: Open a terminal on the grid login node`) remains available; it is simply not the path prompts take.

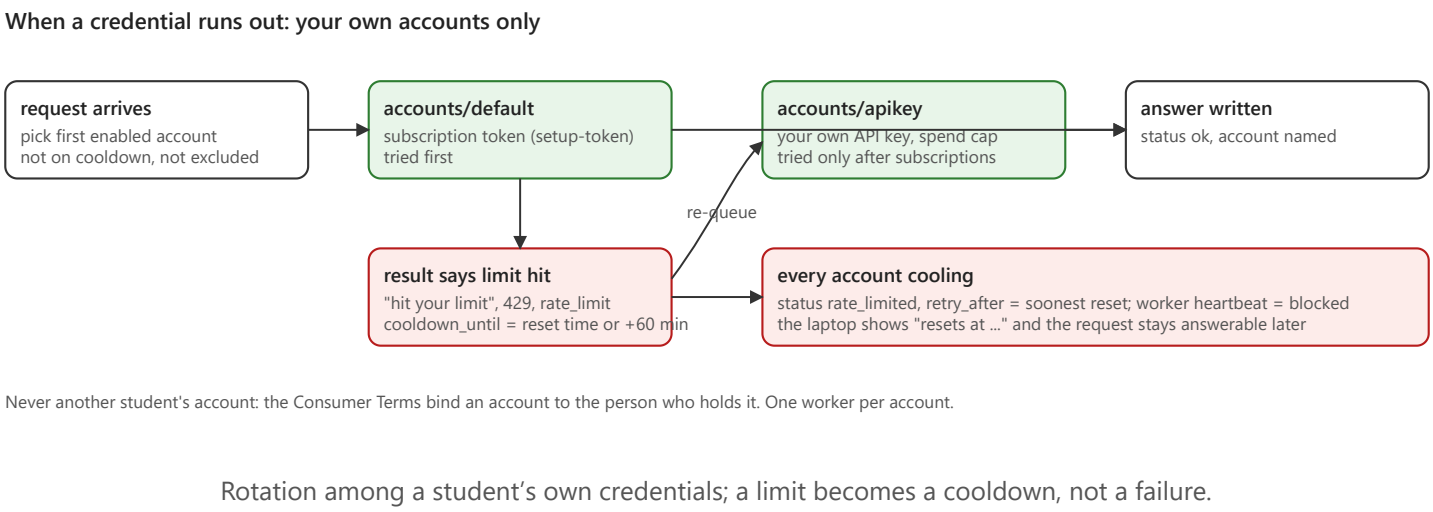
Why not a socket or a TCP port: the laptop can only reach the login node; the daemon runs on whichever node Slurm chose; the shared home is the one thing both see. Files are also what a student can debug with `ls` and `cat`.

Sessions

`~/.ashley/sessions/<name>.json` maps a name the student chose (“thesis”, “figure-2”) to a Claude Code session id, the working directory it is bound to and the account that owns it. The first turn of a session passes `--session-id <uuid>` so the id is known even if the run dies; later turns pass `--resume <uuid>`. Claude Code keeps the transcript under `<CLAUDE_CONFIG_DIR>/projects/<encoded cwd>/<uuid>.jsonl`, which is why a session stays bound to its directory.

`--resume` looks only in the running account’s `projects/`. Measured on Claude Code 2.1.283: from a second config folder the resume answers “No conversation found”; with the transcript present it finds it. A turn moved to the student’s API key after a limit would therefore start from nothing. So every account’s `projects/` is a link to one `~/.ashley/projects` (the daemon links it before each run; `ashley accounts add-token|add-key` link it when an account is made; the diagnostics node `grid.accounts` finds and repairs an unlinked one). All accounts belong to one person, so this shares nothing between people.

Credentials



Rotation among a student’s own credentials; a limit becomes a cooldown, not a failure.

The rule, since version 2: **Ashley never reads, stores, copies or relays a Claude credential.** Anthropic’s Claude Code terms permit an end user to sign in to the unmodified binary with their own subscription, including where a platform hosts it, and forbid a third-party developer to collect, store or intermediate Claude.ai credentials or tokens (`docs/research/reports/`, the compliance report). Version 1 captured the output of `claude setup-token` and copied it to the grid; that was on the wrong side of the line and is gone.

What happens instead:

- `~/.ashley/claude` is a `CLAUDE_CONFIG_DIR` in the student’s own grid account, mode 700. The student runs `ashley claude login` once (the extension opens the terminal): inside their own Slurm allocation, in the lab container, the unmodified `claude auth`

`login` prints its link, the student authorises in the browser and pastes the code, and Claude Code writes its own credential there. Ashley wrote nothing but Claude Code's first-run settings.

- Every Claude Code process Ashley starts on the grid (the worker's `claude -p`, Remote Control) gets `CLAUDE_CONFIG_DIR=~/.ashley/claude` and an environment with every credential variable removed (`CLAUDE_CODE_OAUTH_TOKEN`, `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, `ANTHROPIC_BASE_URL` and the file-descriptor variants). Claude Code uses what it stored.
- The daemon asks `claude auth status` at start, every 30 minutes and when the credential file changes, and puts the answer (signed in or not, the e-mail Claude Code reports) in its heartbeat. The laptop shows it; the diagnostics node `grid.claude-login` reads it; a prompt sent while not signed in is answered with the command that fixes it.
- One person, one Claude account. The only other credential is the student's own Anthropic Console key (`~/.ashley/accounts/apikey/apikey`, Commercial Terms), used only when the subscription window is exhausted, and only in that process's environment.
- On the laptop, the wizard checks `claude auth status` of the student's own Claude Code and requires a subscription sign-in (Pro or Max), because the lab works on subscriptions; it stores nothing.
- A copied `.credentials.json` is never used: a copy shares a refresh-token chain with the machine it came from, and the first refresh on either side revokes the other.

The sign-in lasts about a month (a user report; Anthropic's docs give a three-day warning before expiry). The heartbeat notices when it ends; the extension asks for step 12 again.

Secrets: where they are and where they are not

Secret	Laptop	Grid	Never
GitHub, Hugging Face tokens	VS Code SecretStorage (OS keychain); injected into integrated terminals	`~/.ashley/secrets.env` (mode 600), read by the daemon into the worker's environment	job logs, `sbatch --export`, command lines
Claude subscription token	SecretStorage	`~/.ashley/accounts/default/token` (600)	laptop terminals
Anthropic API key	SecretStorage	`~/.ashley/accounts/apikey/apikey` (600)	laptop terminals (it would silently switch the local Claude Code to pay-per-use)
Grid password / OTP	nowhere	nowhere	typed once, in a terminal, to install the SSH key

Files travel over ssh **stdin** (`cat > file` with `umask 077`), never as arguments.

The container

`bootstrap.sbatch` (one job per user, 7 minutes measured) builds an unprivileged Apptainer **sandbox directory** from `docker://node:22-bookworm` on the student's CephFS space and installs `@anthropic-ai/claude-code@2.1.283` into a persistent npm prefix there. A sandbox rather than a `.sif`: SIF-to-sandbox conversion at exec time is what failed on this grid before, and several nodes lack `unsquashfs`. The host is CentOS 7 (glibc 2.17); Claude Code needs Node 22, which needs a newer glibc, hence the container (glibc 2.36 inside). Runtime order tried: `module load apptainer/1.2.2` (worked on `asx1`), the conda `singularity` 3.8.6, `module load singularity/3.5.2`, `/usr/bin/apptainer`. `~/.local/bin/singularity` is skipped on purpose.

The worker job (`worker.sbatch.tpl`) execs the daemon inside the sandbox with `$HOME`, the CephFS tree and a `/dev/shm` scratch bound to `/tmp` (`/tmp` on nodes is swept by a cleaner while jobs run). `--signal=B:TERM@120` gives the daemon two minutes to finish a turn at the wall limit; `--dependency=singleton` keeps one job per worker name; on the `requeue` QoS the job is `--requeue --open-mode=append` because the daemon is re-entrant.

Slurm inside the worker. The container has no Slurm of its own, so until version 3 the worker's Claude could not submit or even see jobs, although its instructions said it could (found by the dependency study; confirmed on asx1: no `ashley`, `sbatch` or `squeue` on its PATH, and `ashley worker list` called running workers dead). The worker job now binds the host's Slurm client in read-only: `/usr/bin` under `/opt/host/bin`, `/usr/lib64/slurm`, the munge library into `/usr/lib` (a default search

folder, so no `LD_LIBRARY_PATH` leaks into submitted jobs), `/etc/slurm`, the munge socket, and a user list that adds the `slurm` and `munge` users that `slurm.conf` names (without them: "Invalid user for SlurmUser slurm"). Small wrappers in `<worker>/slurm/bin` run the host binaries with the host's PATH and without the container's variables, so a submitted job runs on the host with the student's normal environment. The daemon asks Slurm for its own job at start and puts the answer in its heartbeat; Claude is told either that Slurm works or that it does not, and why. The diagnostics node `grid.slurm-in-worker` reads that answer.

`ashley worker start` saves its settings in `<worker>/params`; `ashley worker restart` stops the job and starts it again from the current template with those settings, so an upgrade of the template reaches running workers.

Grid etiquette, as C&IT stated it on 2026-09-28: nothing container-related runs on the login node, and nothing of ours is left under its `/tmp` (a build there once left 7 GB of `rootfs-*`). `bootstrap.sbatch` refuses to run outside a Slurm job and puts `APPTAINER_TMPDIR`, `APPTAINER_CACHEDIR` and their `SINGULARITY_` twins on the compute node's local disk (`$SLURM_TMPDIR` or `/tmp`), removed by a trap however the job ends; the worker template points the runtime's temp and cache at its `/dev/shm` scratch; `cmd_diag_node` refuses off a job; the diagnostic `grid.login-tmp` finds leftovers under the login node's `/tmp` and removes the recognisable ones on repair. Temp that must be shared between nodes goes under `/wsu/tmp`.

The shared container library

Containers are read-only at run time (every account's sessions, settings and caches live in its own home), so one copy serves the whole lab. `/rs/rs_grp_oschome/shared/ashley/` (group `oschome`, mode 2775) holds `library.json` and one sandbox per entry:

```
{"version": 1, "default": "claude-node22",
 "containers": {"claude-node22": {"sandbox": ".../containers/claude-node22", "claude_bin":
        ".../npm-claude-node22/bin/claude",
   "runtime": "/wsu/el7/gnu7/apptainer/1.2.2/bin/apptainer", "image": "docker://node:22-
        bookworm",
   "claude_version": "2.1.283 (Claude Code)", "gpu": false, "built": "...", "built_by":
        "go2432", "purpose": "general ..."}}}
```

- `ashley install` adopts the library's default when it is readable (`ashley adopt <name>` writes `config.env` and `READY` in a second; nothing is built, nothing is stored in the student's space). `ashley install --build-own` (admin only) builds a private copy for testing an image.
- `ashley worker start --container <name>` runs any entry; the worker's job script carries the sandbox and binds the library root. A student can run a CPU worker on the

general container and a GPU worker on a CUDA one at the same time.

- Leads publish entries with `ashley library publish --name <name> --image docker://<image> [--gpu] [--purpose "..."]`: a job builds the sandbox, installs the pinned Claude Code, merges the entry into `library.json` and makes the tree group-readable. Entries are pinned by image and Claude Code version, so every student runs the same bits; an update is a new entry.
- **The lab's philosophy.** The admin curates one folder, `/rs/rs_grp_oschome/shared`: datasets, Claude containers, software. Students only ever **read** it; all their science runs in their own grid accounts (home, jobs, workers, their own tokens and subscription). Nothing under the lab volume is writable by students, not even a request drop-box.
- **Requests travel as held Slurm jobs.** *Join the lab* (the wizard's last step) submits one held, zero-work job named `ashley_access` whose comment carries the student's note. Slurm authenticates the submitter (`PrivateData=none` here, so the admin sees every user's jobs), so a request cannot be forged or squatted and uses no storage. The admin's ingest writes `status/<id>.json` (mode 600 plus an ACL for that student only) and gives the applicant a traverse-only ACL on the path to it. Decisions: approve, deny or remove with a message, reset (the student may ask again); each goes to `access/requests/decisions.log`. The student's own held job is cancelled once a decision exists.
- **Approval is a read-only ACL** (CephFS honours POSIX ACLs; the admin owns the volume): `--x` on the volume root, `r-x` plus a default ACL on `shared/`, and `r-X` recursively with default ACLs on everything under it except `access/`, walking physically (links are never followed out of the folder). No unix-group change, no ticket. `ashley access admin resync` re-applies grants after data is copied in with `cp -a` or `mv`; publishing a container resyncs automatically.
- **Layout after `ashley access init`:** volume root 2770 with a default ACL `o::---` (new entries start closed); `shared/` and its contents 2750 (the group reads, only the admin writes); `access/` and `access/status/` 2710; `access/requests/` 2700. Nothing is open to other grid users.
- **The request is refused until the environment works the way the lab works:** every earlier step done; the Claude Code and Remote-SSH extensions and the bridge present; and, checked on the grid by `ashley access requirements`, a Claude subscription token the API accepts (`GET /v1/models` with the OAuth token must answer 200; 401 means invalid or revoked; `claude setup-token` only issues tokens to paid plans), and GitHub and Hugging Face tokens that verify. The shared container is what access gives, never a precondition; students are never offered a private build (`ashley install --build-own` is for the admin only).
- **Token checks never put a token on a command line:** `curl -K -` reads the header from stdin, so nothing appears in the login node's process list (sampled during three runs: nothing).

Because every worker of an account shares `~/.ashley`, a worker Claude can hand work to another worker with `ashley send --worker <name>` from its shell; the daemon tells each worker who it is, which peers are alive and the exact command (`peerNote()` in `ashleyd.js`). A lead worker on the general container can therefore fan out to a GPU worker on a CUDA container and collect the result with `ashley result`.

Wayne State sign-in and the pre-approved list

The wizard's first step signs the student in with the university's Microsoft account through VS Code's built-in Microsoft sign-in (work or school accounts only; only the account's e-mail is used). The AccessID is the part before `@wayne.edu`, and the student continues only if it is on the lab's approved list. That list lives in the access service (`tools/intake/worker.js`, a Cloudflare Worker): students ask for access on the website (`docs/site/request.html`: AccessID, name, e-mail, what for), the maintainer approves, denies, adds or removes people on `docs/site/admin.html` (admin token plus Cloudflare Access), and the extension fetches the current list from the service's `/allowlist`, in the same shape as the bundled file. An approved person's trial button asks the service's `/trial` for a key, issued after the service has asked Microsoft who they are; removing a person puts every key issued to them on the signed revocation list the service serves at `/revoked`. The service's addresses are in `docs/site/services.json`. The bundled list, `extension/media/allowlist.json` (`python tools/allow.py add ab1234`, then rebuild), is the fallback when no service is configured there or in `ashley.allowlistUrl`. This is a gate for honest users; the enforceable boundary is still the grid (the student's own account and the admin's approval). The lab assumes the student's Claude, GitHub and Hugging Face accounts were made with AccessID@wayne.edu.

Phone access

Claude Code's Remote Control lets the Claude app (or claude.ai/code) drive a session running elsewhere, over outbound HTTPS only. On the grid the worker hosts it (`grid/phone.js`, inside the worker job). It uses the same sign-in as the worker (`~/.ashley/claude`): Remote Control needs a full login, which is what `ashley claude login` makes. Two things are the student's own acts, in a terminal, because Anthropic's flow and its consent belong to the account holder: the sign-in, and Claude Code's one-time question "Enable Remote Control?", which `ashley claude remote-consent` puts in front of them; their answer is recorded in a marker file and repeated to Claude Code if it asks again. Until both exist, phone access waits and says which one is missing.

Only one worker hosts phone access. All workers share the one sign-in, and two hosts on it would fight. The host holds a lease: a directory `~/.ashley/remote/host.lock` made with `mkdir` (atomic on the shared home, where `flock` is not reliable across nodes), with a beat every 5 s, treated as abandoned after 60 s, the timings Claude Code

uses for its own credential locks. `ashley phone on --worker B` turns phone access off on the other workers; B shows *elsewhere* for about a second and then takes over. Turning phone access off only stops processes: `claude auth logout` would revoke the login, so Ashley never runs it. The handlers act only for the current session, so output from a stopped process can neither change the status nor raise inside the daemon.

Students are told, in the terminal and in the manual, that while the phone is connected the session transcript, tool activity included, is stored on Anthropic's servers; confidential data stays out of phone sessions.

Terms and the signing dialog

Before anything else runs, `vscode/termsPanel.ts` shows the Trial Edition agreement (`LICENSE.txt`) and the privacy statement in a webview: the full text must be scrolled to its end (opening it in an editor also counts), four un-prechecked statements (each names the button) must be ticked, a name typed, and "Accept and Continue" chosen. The record (`core/terms.ts`: terms version, edition, SHA-256 of both texts, each statement's wording and state, the typed name, the surface, the host and product versions, the locale, the install id, the time) is written to `~/.ashley-local/terms.json` with the exact texts beside it, so the person can store and print them (Michigan UETA, MCL 450.838(3)), and the CLI (`ashley-cli terms`) and the MCP server refuse to work without it.

The paid Full Edition has its own agreement (`LICENSE-FULL.txt`), shown by the same dialog when a key whose edition is not `trial` is registered (`licenseGate.set` calls `requireFullTerms`; the CLI refuses such a key until the record exists); its record is `terms-full.json`, and from that moment it governs. A changed terms version asks again; dispute terms never take effect by posting. This is the assent pattern the click-through cases enforce (Berman, Chabolla, In re StockX): conspicuous notice, an affirmative act whose label names the agreement, and a record of what was shown.

Licenses

Every copy of Ashley needs a license key for the person using it (`docs/LICENSING.md`). Keys are offline, Ed25519-signed (`grid/license.js`, one implementation for the extension, the CLI, the MCP server and the grid daemon), name an AccessID or e-mail, carry an edition, dates and a grace period, and are checked against the Wayne State sign-in the extension recorded. The extension refuses every command but the ones that get you a key; the daemon refuses work; both compare the key against a signed revocation list bundled with the release and, where the lab publishes one, fetched daily. A clock set back does not revive a key: the latest time seen is kept. Trials are keys of edition `trial`, one per AccessID, issued by the lab's licence service (`tools/intake/`)

after it has asked Microsoft who the signed-in person is, or by the lead with the issuing tool.

Error reports

With the student's permission, given at first run and changeable at any time, the extension sends the maintainer one report per distinct failure per day after a troubleshooter run that found a failure or made a repair (`extension/src/core/diagReport.ts`, `diagShare.ts`). The report holds versions, the checks' ids and verdicts, and the failing checks' messages with e-mails, AccessIDs, paths, hostnames, addresses, job numbers and token-shaped strings replaced. VS Code's own telemetry setting is honoured; with no report address configured nothing is sent; every report is kept on the laptop for the student to read. On the maintainer's side the service files it as a GitHub issue and the triage workflow (`.github/workflows/triage.yml`) has Claude Code, on the maintainer's own API key, propose a reviewed draft pull request. Nothing in this path touches a student's Claude account.

Diagnostics

Every assumption Ashley makes, from the laptop's OpenSSH to the node's pseudo-terminal, is a node in one dependency graph (`extension/src/vscode/diagNodes.ts`, grid side `ashley diag`). The troubleshooter checks the nodes in dependency order, names the lowest broken one as the cause of everything above it, and runs a repair where one is safe. A feature is not integrated until its nodes are in the graph, and a test fails when a feature's nodes go missing. The map, the rule, and how to add a node: `docs/DIAGNOSTICS.md`.

The laptop side

- **Wizard** (`core/wizard.ts`): ten steps in a strict order (Wayne State sign-in, GitHub, Hugging Face, Claude, grid account, SSH key, SSH config, login, grid install, join the lab), each with `check()` and `run()`; a single step never runs before every earlier one is done; over an abstract `WizardEnv` (settings, secrets, UI). The VS Code extension and the terminal CLI plug in their own UI. Rerunning does only what is missing.
- **SSH config** (`core/sshConfig.ts`): one managed block between markers, inserted first so it wins over any `Host *`. `MACs +hmac-sha1` because the grid's sshd offers nothing newer to Windows OpenSSH 9.5; `AddressFamily inet` because on Windows the IPv6 attempt to the login node times out first (13 s per connection measured, 2 s with IPv4 only); `IdentitiesOnly` so the agent's other keys are not tried; a wildcard `Host *.nas.grid.wayne.edu` with `ProxyJump` so no per-node edit is ever needed. On Windows the same block and key are mirrored into WSL.
- **Interactive steps** open a VS Code terminal whose *process* is `ssh` or `claude` (no shell, no quoting) and read its exit status.

- **Environment**: `environmentVariableCollection` (persistent) rather than editing `terminal.integrated.env` settings, so it can be cleared and does not leak into settings sync.
- **MCP**: `claude mcp add-json --scope user` with the server run by system `node` (>= 20) or by VS Code's own Electron with `ELECTRON_RUN_AS_NODE=1`, so students need not install Node. Fallback: `.mcp.json` in the workspace.

Xcode and other editors

`core/` never imports `vscode`. `dist/ashley-cli.js` runs the same wizard and the same grid calls from a terminal; `cli/ashley` is a ten-line ssh forwarder for machines without Node. The MCP server is editor-agnostic: any client that speaks MCP over stdio (Claude Code in Xcode's terminal, Claude Desktop, an Agent SDK script) can register it. A native Xcode extension is the wrong layer: Xcode Source Editor Extensions cannot open terminals, store secrets or run processes; the CLI plus MCP is the port.

What is deliberately not here

- No Nextflow yet. The queue and the worker are the substrate a pipeline can call (`ashley send` from a Nextflow process, `ashley result` in the next); the recipe's Nextflow section stays the plan.
- No shared lab account, no cross-student pooling.
- No X11. Remote-SSH renders images; `srun --x11` remains available for the rare tool that needs a display.

What you can ask, and what you cannot

Written for students. “The grid Claude” is the Claude Code worker running in your Slurm job with your subscription; “the laptop Claude” is the Claude Code extension in your VS Code.

How the lab works

The lab curates one folder of datasets, Claude containers and software. You read it; you never write to it. Everything you do, you do in your own grid account.

Things that work well

Ask the grid Claude to do work where the data is. It runs in your grid home with your environment: read a dataset’s manifest, write a measurement script, run it on ten records, look at the numbers, fix the script, run it on all 802, write the results file, draft the JOURNAL line. Say which folder (a session is bound to one), which files, and what “done” looks like.

Ask for a Slurm job, not a long foreground run. “Write `slurm/measure.sh` from the lab template, show it to me, and after I say submit, submit it and record the job id.” The worker can submit and poll jobs; it should not itself run a two-hour computation in one turn.

Ask it to continue. Sessions are named. “In session `thesis`, what did we decide about the exclusion list?” picks up where it stopped, days later, on any node.

Ask the laptop Claude to delegate. “Check the grid status, then ask the grid Claude to run the pelvic width demo on ten records and tell me the mean.” The laptop Claude uses the `ashley_*` tools; the heavy part happens on the grid; the answer returns to your panel.

Long tasks. `ashley_send` queues; `ashley_result` collects later. Close the laptop. The job runs until its wall time (24 h by default, 30 days maximum on the `primary` QoS).

From your phone. *Ashley: Phone access* puts your worker’s Claude in the Claude app on your phone (Remote Control), for the same account. One sign-in on the worker, good for about 30 days.

Images. Open your grid files with **Ashley: Open my grid files on a compute node**; click a PNG. No X server.

Containers. Workers run the lab's shared container once your lead has approved your access (the wizard's last step files the request when everything else works; *Ashley: Lab access* shows its state); a lead can publish others (a CUDA one for training, an R/Bioconductor one) and you pick per worker. Two workers of yours can run different containers at once and hand work to each other.

GPUs. Start a worker with one GPU when the task trains or infers; the grid has H200, L40S, A30 and V100-class nodes. Ask for a GPU only for that worker; CPU workers start sooner.

Things that do not work, and why

You cannot	Because
Run the worker or VS Code's server on the login node	C&IT kills processes on `warrior`; Ashley only ever connects VS Code to a compute node and runs a half-second file poll on the login node
Use someone else's Claude account, or pool accounts across students	Anthropic's consumer terms: an account is for the person who holds it. Ashley rotates only *your* credentials (your subscription, then your own API key)
Get more than your plan's window	A Pro/Max window is per account. When it is exhausted the worker reports `rate_limited` with the reset time and waits; an API key with a spending cap is the only overflow
Put patient data, PHI, survey CSVs or contact lists on the grid	The lab's IRB umbrella (HPR 2026-269) covers public, de-identified datasets only; anything else needs its own filing first (RECIPE.md, data rules)
Push the grid Claude to `--dangerously-skip-permissions` for everything	It is available (`mode: bypassPermissions`) but the default `acceptEdits` lets it edit files and run tools while refusing destructive commands; use bypass only inside a project folder you can `git checkout` back
Ask for interactive programs (ITK-SNAP, a GUI) through the worker	The worker is headless. Run GUI tools locally on rsynced results, or with `srun --x11` in your own shell
Expect a reply in under a few seconds	A prompt reaches the worker in about 0.2 s over the open relay, but Claude Code's own start (about 5 s) comes before the first token; a real task takes minutes
Keep a worker forever	Jobs have wall limits; the `requeue` QoS may requeue the job (it resumes). Start another; queued requests wait
Store 200 GB in your home	Home is a 4 TB soft quota shared with everything else; Ashley's container and workspaces live on the lab's CephFS volume when you have access, and the doctor shows the quota

What the worker Claude can and cannot see

Can: your grid home and CephFS space, the internet (PubMed, Hugging Face, GitHub, Zenodo, pip), your GitHub and Hugging Face tokens (as environment variables, for `git push` and `hf download`), the Slurm commands, the lab's Python environments.

Cannot: your laptop's files (it is on the grid), your Claude password (it holds a token you can revoke), other students' homes, the login node's CPUs for anything heavy.

Good prompts

- “In session `pelvis`, working in `~/Student_Projects/projects/lee_pelvic_width`, read `proposal.md`, then write `code/measure_width.py` that measures the bi-iliac width on the label volumes listed in `lists/dev10.txt`, run it, and show me the ten numbers with units.”
- “Write the Slurm script for the full 802 from the lab template. Do not submit; show it.”
- “Compare the distribution in `results/widths.json` with the published values in `docs/normative.md`; flag anything outside the widest published SD.”
- “Append today’s entry to JOURNAL.md: what ran, job ids, what came out, what is next.”

Prompts that will disappoint

- “Do my whole project.” (No question, no data, no bar for failure. Start with PREREG.md.)
- “Make the number come out significant.” (The recipe forbids editing results by hand and so does the worker’s default mode; fix the script, rerun, report what you get.)
- “Download the CTs.” (195 GB into a shared quota, on a request that may be repeated. Ask your lead for the shared path first.)

Installing and building

For a student (two minutes)

1. Install VS Code and, from the Extensions view, **Claude Code** (`anthropic.claude-code`) and **Remote - SSH** (`ms-vscode-remote.remote-ssh`). Sign in to Claude Code with your Claude Pro or Max account.
2. Get the file `ashley-2.0.0.vsix` from your lead (or the repository's `dist/` folder).
3. In VS Code: Extensions view, the `...` menu at the top right, **Install from VSIX...**, pick the file. Or from a terminal: `code --install-extension ashley-2.0.0.vsix`.
4. Command Palette (Ctrl/Cmd+Shift+P): **Ashley: Set up my research environment (wizard)**.

Before the wizard you need, once: a grid account (request it at tech.wayne.edu/kb/high-performance-computing naming your sponsor), and Google Authenticator set up through ondemand.grid.wayne.edu, Interactive Apps, 2FA Setup (VPN required for OnDemand).

What the wizard writes on your laptop

Where	What
`~/.ssh/id_ed25519_wsu`, `.pub`	a key pair without passphrase
`~/.ssh/config`	one block between `# >>> ashley` and `# <<< ashley` (a backup `config.ashley.bak` is kept)
OS keychain (Windows Credential Manager / macOS Keychain / libsecret)	GitHub, Hugging Face, Claude tokens
VS Code user settings `ashley.*`	your AccessID and preferences

On the grid it writes `~/.ashley/` (scripts, tokens with mode 600, workers, sessions) and a block in `~/.bashrc`. The Claude container is the lab's shared one; nothing large is stored in your space.

To undo: remove the block from `~/.ssh/config`, delete the key, run **Ashley: Manage stored tokens** to delete tokens, uninstall the extension; on the grid, `rm -rf ~/.ashley` and the block in `~/.bashrc`.

Emulating a from-scratch install

`Ashley: Reset this laptop (emulate a fresh install)` forgets tokens, key and config block, settings, the Claude Code registration, and optionally Ashley on the grid (`ashley uninstall`, with `--purge-root` for the container). For a brand-new VS Code as well:

```
code --user-data-dir $env:TEMP\fresh-vscode --extensions-dir $env:TEMP\fresh-ext
```

install Claude Code, Remote - SSH and the Ashley VSIX in that window, and run the wizard. Delete the two folders afterwards.

Building from source

Requirements: Node.js 22 (the packaging tool refuses older), Git.

```
git clone <this repository> ashley
cd ashley/extension
npm install
npm run build          # bundles dist/extension.js, dist/mcp-server.js, dist/ashley-cli.js,
         copies ../grid into dist/grid
npm run test:unit      # 22 tests: ssh config, protocol, slurm parsing, and the daemon with a
         fake claude
npm run test:vscode    # downloads a VS Code build once, runs the integration suite
npm run package        # writes ../dist/ashley-<version>.vsix
code --install-extension ../dist/ashley-2.0.0.vsix
```

On a Windows machine without Node on `PATH`, the portable build works: download `node-v22.x-win-x64.zip` from nodejs.org, unzip, and prefix `PATH` with that folder for the commands above.

The extension bundles everything it needs (`vsce package --no-dependencies` plus esbuild), so the `.vsix` has no `node_modules`; students never run `npm`.

The terminal CLI (macOS, Linux, Xcode users)

`extension/dist/ashley-cli.js` is the wizard and the grid calls without VS Code:

```
node dist/ashley-cli.js setup            # the same nine steps, in the terminal
node dist/ashley-cli.js doctor
node dist/ashley-cli.js worker start --name default --hours 24
node dist/ashley-cli.js ask --session thesis "read PREREG.md and list its hypotheses"
```

`cli/ashley` (bash) and `cli/ashley.ps1` forward any command to the grid over ssh and need only an ssh client and the config block (`node dist/ashley-cli.js ssh-config` prints it).

For leads: the shared container library

One group-readable tree on the CephFS volume (`/rs/rs_grp_oschome/shared/ashley`) holds pre-built containers; students adopt them instead of building (no storage used, identical bits for everyone).

```
ssh wsu-grid "ashley library"                                    # what exists, which is the
        default
ssh wsu-grid "ashley library publish --name claude-node22 --image docker://node:22-bookworm --
        purpose 'general: Claude Code, Node 22, git, python3'"
ssh wsu-grid "ashley library publish --name claude-gpu --image docker://node:22-bookworm --gpu -
        -purpose 'GPU: torch and inference'"
```

Each publish is a job (5 to 15 minutes) that builds the sandbox, installs the pinned Claude Code, merges the entry into `library.json` and makes the tree group-readable. The image must already provide Node 22 at `/usr/local/bin/node`, npm and python3: the build checks for Node, and the library entry is written by the container's python3. A bare `nvidia/cuda` image has neither and stops at the check. For GPU work publish a Node 22 image with `--gpu`; a worker started with `--gpus 1` runs it with `--nv`, which brings the host's NVIDIA driver in, and PyTorch's pip wheels carry their own CUDA runtime. The purpose may contain commas.

Who may read it is yours to decide, inside Ashley, with no HPC ticket and no e-mail. Run once:

```
ssh wsu-grid "ashley access init"
```

It closes the volume to other grid users and makes the lab folder read-only for everyone but you. A student reaches **Join the lab**, the wizard's last step, only when everything else works and their Claude (Pro at the minimum), GitHub and Hugging Face tokens verify; the step files one request as a held Slurm job (it never runs; Slurm vouches for who sent it). **Ashley: Lab access** shows you the queue (pending first) with Approve, Deny, Deny with message, Remove, Reset and a direct Grant; the Ashley view shows the pending count. Approve is a read-only POSIX ACL over the lab folder, effective at once and revocable; the student's next worker uses the lab container without building anything. Command-line equivalents:

```
ssh wsu-grid "ashley access admin list"
ssh wsu-grid "ashley access admin approve ab1234"
ssh wsu-grid "ashley access admin deny ab1234 --message 'finish the onboarding first'"
ssh wsu-grid "ashley access admin reset ab1234"      # lets them ask again
ssh wsu-grid "ashley access admin resync"            # after copying data in with cp -a or mv
```

For leads: who may use Ashley at all

The wizard's first step accepts only AccessIDs on the pre-approved list:

```
python tools/allow.py add ab1234 cd5678
cd extension && npm run package      # send students the new .vsix
```

Updating

Install the new `.vsix` the same way; run **Ashley: Install or update Ashley on the grid** to push new grid scripts (a job rebuilds the container only if the pinned Claude Code version changed).

Troubleshooting

The troubleshooter

Beethoven keeps an item in the status bar. It shows a tick when everything works, a warning when something needs attention soon, and the name of the broken piece when something is broken. Click it for four actions:

- **Troubleshoot and repair** walks every piece Beethoven depends on, in order: your laptop, VS Code, the network, your accounts, the grid, the container and the workers. When a piece is broken, the pieces that depend on it are marked *blocked* and name it as the cause, so you see one problem, not twenty. Where a repair is safe it runs the repair and checks again.
- **Show log** opens the full record: every check, its result, and every command Beethoven ran.
- **Investigate with Claude** hands the failing check, its log and the known fixes to Claude Code on your laptop, which searches the web and writes a short report. Nothing is changed.
- **Open last report** shows the last run again.

A quick check (no repairs) runs by itself 45 seconds after VS Code starts and then every hour, so the status bar notices an expired token or a stopped worker before you do. **Beethoven: Check everything (no repairs)** runs the full check by hand.

Beethoven repairs by itself: missing VS Code extensions, the SSH configuration, the scripts on the grid, tokens missing from the grid (it copies them from your laptop), a moved VS Code library or container runtime on the grid, a stopped worker, the Claude Code connection, and unshared session history. It never repairs by itself anything that needs you: signing in, pressing Authorize, typing your grid password, or approval from your lead.

Symptoms

The steps are the ten in *Getting started*.

Symptom	Cause	Fix
Step 1: "not on the lab's pre-approved list"	your lead has not added your AccessID yet	ask your lead; they approve you on the lab's page, then run the step again
Step 1: the Microsoft sign-in does not accept your account	you signed in with a personal Microsoft account	sign in with AccessID@wayne.edu
Step 2: GitHub sign-in does not open a browser	VS Code's GitHub sign-in is disabled or blocked	the panel shows a pasted-token field with the token page pre-filled
Steps 3 or 4: the panel does not notice the copied token	clipboard access blocked, or the copy did not happen	paste it into the field; the check is the same
A token shows *no longer accepted*, or its row in the Beethoven view says *expired*	it expired or was revoked	press **Renew** next to it, or click the row: one click in the browser, and the grid copy is updated too
`no matching MAC found. Their offer: hmac-sha1`	your SSH configuration lacks the Beethoven block	run step 7 again (it writes the block with `MACs +hmac-sha1`)
Every grid action takes 13 s	an IPv6 attempt on Windows	the block sets `AddressFamily inet`; run step 7 again if you edited the file
Step 8 rejects the password or code three times	wrong password or code, a grid account not yet created, or the Authenticator not set up	check that ondemand.grid.wayne.edu logs you in, then run the wizard again. If the panel cannot reach the grid at all, it falls back to a terminal for the same two prompts
`Permission denied (publickey...)` after step 8	the key was not added to `authorized_keys`	run step 8 again; on the grid, `cat ~/.ssh/authorized_keys` should contain "beethoven" (a key made by Ashley says "ashley")
Step 10 (*Join the lab*) refuses	a step above is not finished, or a token does not verify	the panel lists what is missing; fix that step and run the wizard again
Beethoven view: *Waiting for lab access*	your request is pending	your lead approves it in Beethoven; nothing to do
Worker shows PENDING for a long time	the `requeue` pool is busy	start it with the `secp` / `secondary` settings (`beethoven.worker.partition`, `beethoven.worker.qos`)

Symptom	Cause	Fix
A prompt returns *Claude is not signed in on the grid*	you have not done step 12, or the sign-in ended (about a month)	run **Beethoven: Sign in to Claude on the grid**
Step 5 says *signed in with an API key, not a Claude subscription*	Claude Code on your laptop is using a Console key	in Claude Code, sign out and sign in with your Claude Pro or Max account
Beethoven refuses every command and says *license needed*	no key, an ended trial, or a key for someone else	**Beethoven: License**: enter your key, or ask your lead. A trial ends by itself and cannot be repeated
Beethoven says the terms have not been accepted	the first-run dialog was closed with *Not now*	run any Beethoven command; the dialog returns
Phone access says *need_consent*	Claude Code's one-time Remote Control question has not been answered	**Beethoven: Phone access** opens the terminal; type y when asked
A prompt returns `rate_limited` with a time	your subscription window is used up	wait for the time shown; Beethoven never switches to another person's account
Phone access: *Claude did not accept that code*	only part of the code was copied	press Copy on the claude.ai page (the code is long) and send it again
Phone access: *hosted by worker …*	phone access is on for another of your workers	run **Beethoven: Phone access** and pick this worker; it moves, and the other one turns off
Phone access worked yesterday, not today	the worker job ended, or the 30-day sign-in expired	start a worker; run **Beethoven: Phone access** again
Remote-SSH: *Connecting with SSH timed out*	the compute node's sshd needs about 25 s per session; the first connection also downloads the server	Beethoven sets `remote.SSH.connectTimeout` to 300 when it opens a node; if you connect by hand, set it in Settings and connect again
Remote-SSH: *You are connecting to an OS version that is unsupported by Visual Studio Code*	the grid is CentOS 7; VS Code shows this on every such connection even when the compatibility libraries make it work	dismiss it, or *Don't show again*
Remote-SSH: *glibc* or *exit code 207*	the C&IT compatibility variables are not in `~/.bashrc`, or a `return` line comes before them	**Troubleshoot and repair** rewrites the block; move it above any `[ -z "$PS1" ] && return`
VS Code connects to `warrior` instead of a node	no job was running	start a worker; Beethoven connects only to nodes where you hold a job

Symptom	Cause	Fix
`home quota` above 90 %	your home is full	large files belong in your project space; ask your lead
The MCP tools do not appear in Claude Code	the bridge was not registered, or Claude Code was open before registration	**Troubleshoot and repair** registers it; start a new conversation. The tools are named `beethoven_*` (they were `ashley_*` before the rename)
Install on the grid: *a Claude worker … is running on the grid*	your grid folder gets its new name (`~/.ashley` becomes `~/.beethoven`) once, while none of your workers runs	**Beethoven: Stop a Claude worker**, then run the step again; the worker's queued requests wait in its inbox
The grid scripts keep going back to an older version	Ashley for VS Code is still installed beside Beethoven, and each installs its own scripts	uninstall Ashley for VS Code, then run **Install on the grid** once more
After the upgrade, a token row says *missing*	VS Code keeps each extension's stored tokens apart, so the tokens Ashley stored are not visible to Beethoven	run the wizard's GitHub and Hugging Face steps once (or click the row)
Claude Code lists an MCP server `ashley` that fails to connect	Ashley for VS Code registered it and was uninstalled afterwards; the entry points at its removed folder	run **Beethoven: Connect the Claude Code extension to the grid** once (it removes the `ashley` entry when Ashley is not installed)

Logs

- Laptop: the status bar item's **Show log** (the same as *View, Output, Beethoven*): every check and every command with its exit code. Each troubleshooter run is also saved as a report.
- Grid: `~/.beethoven/workers/<name>/log/beethovend.log`, `~/.beethoven/workers/<name>/log/slurm_<job>.out`, every request's `outbox/<id>.events.jsonl`, and for phone access `rc/session.log` in the worker's folder.
- Remote-SSH: *View, Output, Remote - SSH*.

Undo

Laptop: remove the block from `~/.ssh/config` (a `config.beethoven.bak` is kept), delete `~/.ssh/id_ed25519_wsu*`, **Beethoven: Manage stored tokens** to delete tokens, uninstall the extension. Grid: `~/.beethoven/bin/beethoven uninstall` stops your workers and removes `~/.beethoven`, the `~/.ashley` link and the block in `~/.bashrc`.

Beethoven: Reset does this for you: you pick what to forget, and it asks once before doing it.

Appendix

- `docs/RESEARCH.md`: the research behind the design (Claude Code headless mode, subscription limits and terms, containers on this grid, multi-account tools, orchestration patterns, VS Code Remote-SSH on old glibc, Xcode, literature APIs), with sources and which findings were independently verified.
- `docs/DIAGNOSTICS.md`: the troubleshooter's dependency graph, the rule that every feature adds its checks before it is integrated, and what the dependency study found and fixed.
- `docs/DEPENDENCY_MAP.md`: every assumption Beethoven makes (370 entries), with the check, what a failure looks like and how to fix it; search it for an error message the troubleshooter does not name.
- `docs/research/claude_swap_notes.md`: what Beethoven took from claude-swap and cswap-desktop, and the plan for the next release.
- `docs/CHECKPOINT_2026-09-28.md`: the state of the project on the day this manual was built, including what was tested and what was not.
- `Student_Projects/RECIPE.md`: the lab's method; Beethoven is the on-ramp to it.
- C&IT: "How to SSH to the Grid in Visual Studio Code" and "How do I set up and use Google Authenticator for the Grid" (services.wayne.edu knowledge base).